

Operating Systems

2520030042

K.Ashish

SEC-5

WEEK-4

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code

```
Ashish@Ashish: ~
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t children[3];
    int i, status;

    for (i = 0; i < 3; i++) {
        children[i] = fork();

        if (children[i] < 0) {
            perror("Fork failed");
            return 1;
        }

        if (children[i] == 0) {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(i + 1);
            printf("Child %d completed.\n", i + 1);
            exit(10 + i);
        }
    }

    printf("\nParent using wait()...\n");
    wait(&status);
    printf("One child completed using wait()...\n");

    printf("\nParent using waitpid()...\n");
    for (i = 0; i < 3; i++) {
        waitpid(children[i], &status, 0);

        if (WIFEXITED(status)) {
            printf("Child PID %d exited with status %d\n",
                children[i], WEXITSTATUS(status));
        }
    }

    printf("All children completed.\n");

    return 0;
}
```

output

```
Ashish@Ashish:~$ nano child.c
Ashish@Ashish:~$ gcc child.c
Ashish@Ashish:~$ ./a.out
Child 1: PID = 3859

Parent using wait()...
Child 2: PID = 3860
Child 3: PID = 3861
Child 1 completed.
One child completed using wait().

Parent using waitpid()...
Child PID 3859 exited with status 10
Child 2 completed.
Child PID 3860 exited with status 11
Child 3 completed.
Child PID 3861 exited with status 12
All children completed.
Ashish@Ashish:~$
```